



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Análisis y Diseño de Software I	Unidad 6	Tema: Conceptos OO. Persistencia y Modelo Conceptual.
	Semana 15	Instructora: Katya Michelle Asencio Bernal

Introducción:

Bienvenido al primer material de lectura de la Unidad #6 de Análisis y Diseño de Software I. En este módulo se estudiarán los principales conceptos de la Programación Orientada a Objetos, incluyendo atributos, métodos, encapsulamiento, herencia, polimorfismo y abstracción. Asimismo, se analizará la importancia de la persistencia de datos y su relación con el modelo conceptual, las bases de datos y el mapeo objeto-relacional (ORM), elementos fundamentales para el diseño y desarrollo de sistemas de software.

Este material de lectura busca brindarte una base sólida en campo del Análisis y Diseño de Software, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas.

Sección 1:

6.2.2 Atributos y Métodos

Atributos

Los atributos son las características, propiedades o datos que describen el estado de un objeto. Representan la información que una clase almacena sobre las entidades que modela.

Cuando se crea un objeto a partir de una clase, cada atributo contiene un valor específico que permite diferenciar a un objeto de otro.

Características de los atributos

- Definen el estado de un objeto.
- Se declaran dentro de una clase.
- Pueden tener diferentes tipos de datos (entero, decimal, cadena, booleano, etc.).
- Pueden tener distintos niveles de acceso (público, privado o protegido).
- Cada objeto posee sus propios valores para los atributos definidos en la clase.

Métodos

Los métodos son las acciones, comportamientos u operaciones que puede realizar un objeto. Contienen el código que define como un objeto interactúa con otros objetos o modifica su propio estado.

Los métodos permiten manipular los atributos y ejecutar funcionalidades específicas.

Características de los métodos

- Definen el comportamiento de los objetos.
- Pueden recibir parámetros de entrada.
- Pueden devolver resultados.
- Pueden modificar los valores de los atributos.



Actividad Interactiva #1

Identificando atributos

Observa la siguiente clase:

Empleado

Escribe tres atributos que podrían pertenecer a esta clase.

1. _____

2. _____

3. _____

- Permiten implementar las funcionalidades del sistema.

6.2.3 Encapsulamiento

La encapsulación no es un concepto propio de la programación orientada a objetos, pero es fundamental, los objetos son la abstracción que proporciona la encapsulación.

La encapsulación consiste en hacer que los datos sean modificados únicamente por las funciones destinadas a tal efecto. La encapsulación permite que los datos conserven un estado válido y consistente, trata de evitar que cualquier código pueda modificar una estructura de datos con el consiguiente problema de generación de inconsistencias.

Se denomina encapsulación porque los datos y sus estructuras de datos no están accesibles de forma directa, sino que para acceder a los datos o manipularlos se ha de realizar a través de las funciones asociadas, los datos están encapsulados.

6.2.4 Herencia

La herencia es uno de los diferenciadores clave entre sistemas convencionales y orientados a objeto. Una subclase Y hereda todos los atributos y operaciones asociadas con su superclase X. Esto significa que todas las estructuras de datos y algoritmos originalmente diseñados e implementados para X están disponibles de inmediato para Y; no se necesita hacer más trabajo. La reutilización se logra directamente.

Cualquier cambio a los atributos u operaciones contenidos dentro de una superclase se hereda inmediatamente para todas las subclases. Por tanto, la jerarquía de clase se convierte en un mecanismo mediante el cual los cambios (en niveles superiores) pueden propagarse inmediatamente a través de un sistema.

Es importante observar que en cada nivel de la jerarquía de clase pueden agregarse nuevos atributos y operaciones a las que se heredaron de niveles superiores en la jerarquía. De hecho, siempre que se crea una nueva clase, tendrá algunas opciones:

- La clase puede diseñarse y construirse desde cero, es decir, no se usa herencia.
- La jerarquía de clase puede revisarse para determinar si una clase superior en la jerarquía contiene más de los atributos y operaciones requeridos. La nueva clase hereda de la clase superior y entonces pueden agregarse adiciones, según se requiera.
- La jerarquía de clase puede reestructurarse de modo que los atributos y operaciones requeridos pueden heredarse en la nueva clase.

- Las características de una clase existente pueden ser excesivas, y diferentes versiones de atributos u operaciones se implementan para la nueva clase.

Como todo concepto de diseño fundamental, la herencia puede proporcionar beneficios significativos para el diseño, pero si se usa de manera inadecuada, puede complicar un diseño de manera innecesaria y conducir a software proclive a errores, que es difícil de mantener.

6.2.5 Polimorfismo

El polimorfismo (que significa muchas formas) o redefinición de métodos (no es lo mismo que sobrecarga de métodos) es la capacidad de un programa orientado a objetos de tener varias versiones de un mismo método con el mismo nombre dentro de una relación superclase/subclase. La subclase hereda un método padre, pero le puede agregar elementos o modificarlo. La subclase puede cambiar el tipo de los datos, o puede cambiar la forma en que trabaja el método. Por ejemplo, podría haber un cliente que reciba un descuento adicional por volumen, con lo que se usaría un método modificado para calcular el total del pedido. Se dice que el método de la subclase redefine al método de la superclase.

Cuando los atributos o métodos se definen más de una vez, se utiliza el más específico (el más bajo en la jerarquía de clases). El programa compilado recorre la cadena de clases en busca de los métodos.

6.2.6 Abstracción

La abstracción es el concepto por el que un modelo es creado con las propiedades relevantes a observar. Un programa trata únicamente con las propiedades de un objeto que al programa le interesa. Las clases son la abstracción de los conceptos que maneja la aplicación, pueden ser conceptos que existan en el mundo real pero simplificados al tener únicamente las propiedades relevantes para la aplicación. Las clases también pueden ser conceptos que no tengan una existencia física en el mundo real como una lista de elementos, una dirección IP o un archivo de ordenador.

Un avión es un objeto físico del mundo real con multitud de propiedades, desde su fabricante y modelo, color, tamaño, número de asientos, ubicación, capacidad de carga, peso, año de diseño y fabricación, materiales de fabricación, altitud, posición GPS, dirección, velocidad y distancia máxima y muchas otras. De todas estas propiedades en una aplicación de gestión de embarque le interesará únicamente las propiedades de los asientos, quizá en otra aplicación para la programación de vuelo le interesa otras propiedades como altitud, posición GPS, dirección, velocidad aeropuerto origen y destino o distancia.

Sección 2:

6.3 Persistencia y el Modelo Conceptual

La persistencia se refiere a la capacidad de almacenar y recuperar datos a lo largo del tiempo. En el contexto del desarrollo de software, esto implica cómo los datos se guardan y recuperan de manera eficiente y confiable en una aplicación. Por otro lado, el modelo conceptual se refiere a la representación abstracta y organizada de los conceptos clave y las relaciones entre ellos en un sistema.

La relación entre persistencia y modelo conceptual es crucial en el desarrollo de software por varias razones:

- **Integridad de Datos:** Un modelo conceptual sólido proporciona una estructura clara y coherente para los datos en una aplicación. La persistencia debe ser diseñada de manera que conserve la integridad de estos datos a lo largo del tiempo, garantizando que la información almacenada cumpla con la estructura y las restricciones definidas por el modelo conceptual.
- **Eficiencia en el Acceso a Datos:** La forma en que se diseñan los mecanismos de persistencia puede afectar significativamente el rendimiento de la aplicación. Un modelo conceptual bien diseñado puede ayudar a optimizar la manera en que se accede a los datos, permitiendo consultas y operaciones eficientes. La persistencia debe estar alineada con este modelo para garantizar un acceso rápido y eficiente a la información almacenada.
- **Mantenimiento y Evolución del Software:** Los modelos conceptuales proporcionan una guía clara sobre cómo se organizan y relacionan los datos en una aplicación. Cuando se realizan cambios en el software, ya sea para agregar nuevas características o corregir errores, la persistencia debe adaptarse para reflejar estos cambios en el modelo conceptual, manteniendo la consistencia y evitando problemas de integridad.
- **Escalabilidad:** A medida que una aplicación crece en tamaño y complejidad, la persistencia debe ser capaz de escalar para manejar mayores volúmenes de datos sin comprometer la coherencia del modelo conceptual. Un diseño de persistencia adecuado debe ser capaz de escalar de manera eficiente con el modelo conceptual subyacente.

La relación entre persistencia y modelo conceptual en el desarrollo de software es esencial para garantizar la coherencia, integridad y eficiencia en el manejo de datos a lo largo del ciclo de vida de una aplicación. Un buen diseño en ambas áreas contribuirá a un sistema robusto y fácil de mantener.

[illegible]

denormalización de datos para consultas más eficientes, la indexación adecuada y la gestión de transacciones.

8. **Manejo de Transacciones:** Define cómo se manejarán las transacciones en tu sistema. Asegúrate de que las operaciones de lectura y escritura sean consistentes y seguras.
9. **Pruebas de Persistencia:** Realiza pruebas para asegurarte de que la persistencia funciona como se espera. Esto incluye pruebas de creación, lectura, actualización y eliminación de datos.
10. **Refinamiento Continuo:** A medida que el desarrollo avanza, revisa y ajusta la arquitectura de clases persistentes según sea necesario. La evolución del sistema puede requerir cambios en la estructura de datos.

Este proceso puede variar dependiendo de la tecnología que estés utilizando y de los requisitos específicos del sistema. Es crucial equilibrar la simplicidad con la flexibilidad y la eficiencia en el diseño de la arquitectura de clases persistentes.



6.3.2 Modelo Conceptual y Base de Datos

El modelo conceptual y la base de datos son elementos estrechamente relacionados dentro del proceso de desarrollo de software, ya que ambos tienen como objetivo representar y gestionar la información de un sistema. Sin embargo, cada uno cumple una función diferente dentro del ciclo de análisis y diseño.

El modelo conceptual constituye el punto de partida para la construcción de una base de datos. Durante esta etapa se identifican las entidades relevantes del negocio, sus atributos y las relaciones que existen entre ellas. Su propósito es representar la realidad del dominio de manera clara y comprensible para los usuarios y analistas, sin preocuparse todavía por aspectos técnicos de implementación.

Una vez definido el modelo conceptual, este sirve como guía para diseñar la estructura de la base de datos. Las entidades identificadas se convierten generalmente en tablas, los atributos en columnas y las relaciones en claves primarias y foráneas que permiten mantener la integridad de los datos. De esta manera, la información modelada conceptualmente puede almacenarse de forma organizada y eficiente.

Además, una buena relación entre el modelo conceptual y la base de datos permite:

- Mantener la coherencia entre los requerimientos del negocio y la implementación técnica.
- Reducir errores de diseño durante el desarrollo.
- Facilitar el mantenimiento y la evolución del sistema.
- Garantizar que la información almacenada responda a las necesidades reales de los usuarios.
- Mejorar la calidad y consistencia de los datos.

Es importante destacar que la base de datos no debería diseñarse directamente sin un modelo conceptual previo, ya que esto puede provocar estructuras desorganizadas, redundancia de información y dificultades para representar correctamente las reglas del negocio.

6.3.3 Relación entre clases y tablas

La relación entre clases y tablas surge cuando se integra la Programación Orientada a Objetos (POO) con las bases de datos. Aunque ambos conceptos pertenecen a ámbitos diferentes, comparten el objetivo de representar y organizar información sobre las entidades de un sistema.

En la POO, una clase define la estructura y el comportamiento de los objetos mediante atributos y métodos. En las bases de datos, una tabla almacena información relacionada con una entidad específica mediante columnas y registros. Debido a esta similitud, es común que las clases del sistema se correspondan con las tablas de la base de datos.

A continuación, veremos una tabla de correspondencia entre Clases y Tablas:

POO	BASE DE DATOS RELACIONAL
CLASE	TABLA
OBJETO	REGISTRO (FILA)
ATRIBUTO	COLUMNA
INSTANCIA	REGISTRO ALMACENADO
IDENTIFICADOR DEL OBJETO	CLAVE PRIMARIA
RELACIÓN ENTRE CLASES	CLAVE FÓRNEA O TABLA INTERMEDIA

La relación entre clases y tablas es fundamental porque permite que los objetos creados dentro de una aplicación puedan almacenarse y recuperarse desde una base de datos. Este proceso es conocido como persistencia de datos.

Gracias a esta correspondencia, la información manejada por los objetos no se pierde cuando la aplicación finaliza su ejecución, sino que permanece almacenada de forma permanente en la base de datos.

Asimismo, esta relación facilita el uso de herramientas conocidas como ORM (Object-Relational Mapping), las cuales permiten mapear automáticamente clases y objetos a tablas y registros.

6.3.4 Introducción al Mapeo Objeto-Relacional

El mapeo objeto-relacional (ORM) es el proceso de abstraer la conexión entre las entidades (objetos) del lenguaje de programación y sus correspondientes elementos de base de datos.

Al crear aplicaciones, los programadores definen objetos en código, que corresponden a conceptos del mundo real. Los datos de los objetos se manipulan en código, pero se almacenan en una base de datos tal como se define en el diseño del sistema de datos. Por ejemplo, puede tener un objeto de cliente que contenga información del cliente en su código, pero la base de datos subyacente podría almacenar esos datos en varias tablas.

El mapeo objeto-relacional es una capa de software que traduce los datos de los objetos a la base de datos subyacente, abstrayendo los detalles de la base de datos del programador. De esta forma, el código permanece inalterado incluso si las bases de datos subyacentes se intercambian por diseños nuevos o diferentes. El mapeo objeto-relacional permite la creación de aplicaciones modulares que son fáciles de administrar y mantener.

¿Cómo funciona el ORM?

El mapeo objeto-relacional (ORM) conecta las aplicaciones escritas en un lenguaje de programación orientada a objetos (OOP) a una base de datos subyacente. Estos tipos de aplicaciones y bases de datos a menudo se usan juntas, pero cada una maneja los datos de manera diferente. Agregar un mapeo objeto-relacional entre ellos permite que cada uno envíe datos en un formato que el otro comprenda. Puede modificar una sin afectar a la otra.

Puede crear un mapeo objeto-relacional, a veces denominada administrador objeto-relacional, que indique a la base de datos dónde ubicar todos los datos solicitados por una aplicación, aunque cada estructura almacene sus datos de una manera diferente. Cuando su aplicación solicita todos los atributos de un automóvil individual, el ORM indica a la base de datos cómo recuperar todos esos datos en varias tablas.

Como las asignaciones son abstractas, si la estructura de la base de datos cambia alguna vez o si se migra a una nueva base de datos, el ORM puede seguir apuntando a los datos correctos con actualizaciones mínimas.

¿Cuáles son los beneficios de usar ORM?

Los sistemas ORM brindan los siguientes beneficios:

Desarrollo de aplicaciones más rápido:

Los ORM ayudan a recuperar datos complejos mediante código comprensible en el lenguaje OOP con el que los desarrolladores están más familiarizados. Pueden gestionar las operaciones rutinarias de las bases de datos, como la creación, la lectura, la actualización y la eliminación de registros (CRUD), de modo que los desarrolladores no tengan que codificar estas operaciones repetidamente en la aplicación.

Mantenimiento de aplicaciones más sencillo:

Al resumir las consultas de la base de datos a través de un ORM, los desarrolladores pueden trabajar en un solo idioma, lo que hace que su código sea más fácil de entender y mantener.

Seguridad mejorada:

Un ataque de inyección de código SQL es un intento malintencionado de acceder a datos privados mediante el envío de consultas SQL a través de los formularios de usuario de la aplicación. Los ORM utilizan consultas parametrizadas, que primero envían a la base de datos una consulta con valores de marcador de posición, no con datos reales. (por ejemplo, ? o :param). Los datos reales se envían por separado. Esto garantiza que la entrada del usuario se trate como datos, no como código SQL ejecutable. El uso de una herramienta ORM evita así el acceso no autorizado a los datos mediante la inyección de código SQL.

Mejor rendimiento:

Un ORM puede mejorar el rendimiento del sistema al permitir el almacenamiento en caché de objetos transparente en el nivel de aplicación. Puede guardar los datos que se recuperan con más frecuencia cerca de la aplicación para un acceso más rápido.

Bibliografía

Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas*. Pearson

Educación.

picodotdev. (2021, 03 31). *Los conceptos de encapsulación, herencia, polimorfismo y*

composición de la programación orientada a objetos. picodotdev.

[https://picodotdev.github.io/blog-bitix/2021/03/los-conceptos-de-](https://picodotdev.github.io/blog-bitix/2021/03/los-conceptos-de-encapsulacion-herencia-polimorfismo-y-composicion-de-la-programacion-orientada-a-objetos/#encapsulaci%C3%B3n)

[encapsulacion-herencia-polimorfismo-y-composicion-de-la-programacion-](https://picodotdev.github.io/blog-bitix/2021/03/los-conceptos-de-encapsulacion-herencia-polimorfismo-y-composicion-de-la-programacion-orientada-a-objetos/#encapsulaci%C3%B3n)

[orientada-a-objetos/#encapsulaci%C3%B3n](https://picodotdev.github.io/blog-bitix/2021/03/los-conceptos-de-encapsulacion-herencia-polimorfismo-y-composicion-de-la-programacion-orientada-a-objetos/#encapsulaci%C3%B3n)

Pressman, R. S. (2010). *INGENIERIA DE SOFTWARE*. McGraw-Hill Interamericana de España S.L.

What is Object-Relational Mapping (ORM)? - Object-Relational Mapping Explained.

(n.d.). AWS. <https://aws.amazon.com/what-is/object-relational-mapping/>